

ACTIVIDAD 2

Estructura de datos

Docente: Alberto Lemus Mera



ÍNDICE

Objetivo.....	1
Instrucciones.....	1
Código.....	1
Node.java.....	1
Lista enlazada.java.....	2
Proceso.java.....	6
Contactos.java.....	6
TiposDeDatos.java.....	7
Cola.java.....	8
Pila.java.....	9
App.java.....	10
Código corriendo.....	13

Objetivo

- Hacer uso de listas ligadas para implementar pilas y colas.
- Aplicar estas estructuras en un contexto de simulación de gestión de tareas.
- Fortalecer la habilidad de programación en Java.

Instrucciones

1. Revisa y ajusta el código de listas ligadas entregado en la Actividad 1, asegurándose de que la implementación sea adecuada para ser utilizada en las estructuras de pilas y colas.
2. Implementación de pilas y colas:
 - a. Implementa las clases Pila y Cola utilizando las listas ligadas.
 - b. Codifica métodos para operaciones básicas como agregar (Push/Enqueue), remover elementos (Pop/Dequeue) y verificar el tope de la Pila o de la Cola (Peek).
3. Integración y pruebas:
 - a. Integra las estructuras de datos en un módulo de gestión de sistema operativo simulado.
 - b. Realiza pruebas para verificar la funcionalidad y detectar errores en el código.

Código

Node.java

```
public class Node<T> {
    T data;
    Node<T> next;
    Node<T> prev;

    public Node(T data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }

    public String toString() {
        return data != null ? data.toString() : "null";
    }
}
```

Lista enlazada.java

```
public class ListaEnlazada<T> {
    public Node<T> head;
    public Node<T> tail; // Listas circulares
    public int tipoLista; // 1. Simples 2. Doble 3. Circulares
    public int size;

    // Constructor
    public ListaEnlazada(int tipoLista) {
        this.tipoLista = tipoLista;
        this.head = null;
        this.tail = null;
        this.size = 0;
    }

    // Revisar si está vacío
    public boolean isEmpty() {
        return head == null;
    }

    // Método para Pila
    public void addFirst(T data) {
        Node<T> nuevo = new Node<>(data);
        if (isEmpty()) {
            head = nuevo;
            tail = nuevo;
            if (tipoLista == 3) head.next = head;
        } else {
            if (tipoLista == 1) {
                nuevo.next = head;
                head = nuevo;
            } else if (tipoLista == 2) {
                nuevo.next = head;
                head.prev = nuevo;
                head = nuevo;
            } else if (tipoLista == 3) {
                nuevo.next = head;
                head = nuevo;
                tail.next = head;
            }
        }
        size++;
    }

    // Metodo Pop
    public T removeFirst() {
        if (isEmpty()) return null;

        T dato = head.data;
        if (size == 1) {
            head = null;
            tail = null;
        } else {
            head = head.next;
            if (tipoLista == 2) head.prev = null;
        }
    }
}
```

```
        if (tipoLista == 3) tail.next = head;
    }
    size--;
    return dato;
}

// Metodo Peek
public T getFirst() {
    if (isEmpty()) return null;
    return head.data;
}

// Meter datos para Cola
public void appendNode(T data) {
    Node<T> nuevo = new Node<>(data);
    if (isEmpty()) {
        head = nuevo;
        tail = nuevo;
        if (tipoLista == 3) head.next = head;
    } else {
        if (tipoLista == 1 ) {
            Node<T> actual = head;
            while (actual.next != null) {
                actual = actual.next;
            }
            actual.next = nuevo;
            tail = nuevo;
        } else if (tipoLista == 2) {
            tail.next = nuevo;
            nuevo.prev = tail;
            tail = nuevo;
        } else if (tipoLista == 3) {
            tail.next = nuevo;
            tail = nuevo;
            nuevo.next = head;
        }
    }
    size++;
}

// 3. Mostrar los elementos de las listas
public void showlist() {
    if (isEmpty()) {
        System.out.println("La lista esta vacia");
        return;
    }
    Node <T> actual = head;
    if (tipoLista == 2) System.out.print("null <- ");
    do {
        System.out.print(actual.data + " -> ");
        actual = actual.next;
        if (tipoLista == 3 && actual == head) break;
    } while (actual != null);
}
```

```
if (tipoLista == 3) {
    System.out.println("Fin de la lista circular");
} else {
    System.out.println("null");
}
}

// 4. Buscar elementos
public boolean search(T data) {
    if (isEmpty()) return false;
    Node <T> actual = head;
    do {
        if (actual.data.equals(data)) return true;
        actual = actual.next;
        if (tipoLista == 3 && actual == head) break;
    } while (actual != null);
    return false;
}

// 5. Eliminar algun dato
public void delete(T data) {
    if (isEmpty()) {
        System.out.println("La lista esta vacia");
        return;
    }
    Node<T> actual = head;
    Node<T> anterior = null;
    boolean encontrado = false;

    do {
        if (actual.data.equals(data)) {
            encontrado = true;
            break;
        }
        anterior = actual;
        actual = actual.next;
        if (tipoLista == 3 && actual == head) break;
    } while (actual != null);

    if (!encontrado) {
        System.out.println("Elemento no encontrado");
        return;
    }

    if (size == 1) {
        head = null;
        tail = null;
    } else if (actual == head) {
        head = head.next;
        if (tipoLista == 2) head.prev = null;
        if (tipoLista == 3) tail.next = head;
    } else if (actual == tail) {
        tail = anterior;
        if (tipoLista == 1 || tipoLista == 2) tail.next = null;
```

```
        if (tipoLista == 3) tail.next = head;
    } else {
        anterior.next = actual.next;
        if (tipoLista == 2 && actual.next != null) {
            actual.next.prev = anterior;
        }
    }
    size--;
    System.out.println("Elemento eliminado");
}
}
```

Proceso.java

```
public class Proceso {
    public int id;
    public String nombre;
    public String usuario;
    public int tiempoEstimado;

    public Proceso(int id, String nombre, String usuario, int
tiempoEstimado) {
        this.id = id;
        this.nombre = nombre;
        this.usuario = usuario;
        this.tiempoEstimado = tiempoEstimado;
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getUsuario() {
        return usuario;
    }

    public int getTiempoEstimado() {
        return tiempoEstimado;
    }

    @Override
    public String toString() {
        return "Id: " + id + "| Proceso: " + nombre + "| Usuario: " +
usuario + "| tiempo estimado: " + tiempoEstimado;
    }
}
```

Contactos.java

```
public class Contactos {
    public String nombre;
    public String direccion;
    public String telefono;

    public Contactos(String nombre, String direccion, String telefono) {
        this.nombre = nombre;
        this.direccion = direccion;
        this.telefono = telefono;
    }

    public String getNombre() {
        return nombre;
    }
    public String getDireccion() {
        return direccion;
    }
    public String getTelefono() {
        return telefono;
    }

    @Override
    public String toString(){
        return "Contacto: " + nombre + ", Direccion: " + direccion + ",
telefono: " + telefono;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;
        Contactos contacto = (Contactos) obj;
        return nombre.trim().equalsIgnoreCase(contacto.nombre.trim());
    }
}
```

TiposDeDatos.java

```
public class tiposDeDatos {
    public static void demostrarTipos() {
        System.out.println("\nDemostracion de diferentes tipos de datos y
listas");
        // 1. Tipos de datos primitivos
        ListaEnlazada<Integer> listaPrimitivos = new ListaEnlazada<>(1);
        listaPrimitivos.appendNode(1);
        listaPrimitivos.appendNode(2);
        listaPrimitivos.appendNode(3);
        System.out.print("Lista de numeros enteros: ");
        listaPrimitivos.showlist();
    }
}
```

```
// 2. Tipos de datos complejos
ListaEnlazada<String> listaComplejos = new ListaEnlazada<>(2);
listaComplejos.appendNode("Hola");
listaComplejos.appendNode("Buenas tardes");
listaComplejos.appendNode("Adios");
System.out.print("Lista de palabras: ");
listaComplejos.showlist();
// 3. Tipos abstractos
ListaEnlazada<Contactos> listaAbstractos = new ListaEnlazada<>(3);
listaAbstractos.appendNode(new Contactos("Erick", "Felipe Sevilla
213", "4424635378"));
listaAbstractos.appendNode(new Contactos("Vale", "Palermo 107",
"4424777457"));
listaAbstractos.appendNode(new Contactos("Daisy", "1 de Mayo",
"4424644984"));
System.out.print("Lista datos abstractos: ");
listaAbstractos.showlist();
}
}
```

Cola.java

```
public class Cola<T> {
    private ListaEnlazada<T> lista;

    public Cola() {
        this.lista = new ListaEnlazada<>(1);
    }

    public boolean isEmpty() {
        return lista.isEmpty();
    }

    public void enqueue(T data) {
        lista.appendNode(data);
    }

    public T dequeue() {
        if (isEmpty()) {
            System.out.println("Queue underflow: Nada que mostrar");
            return null;
        }
        return lista.removeFirst();
    }

    public T peek() {
        if (isEmpty()) {
            System.out.println("Queue underflow: Nada que mostrar");
            return null;
        }
        return lista.getFirst();
    }
}
```

```
}

public int getSize() {
    return lista.size;
}

public void mostrarCola() {
    if (isEmpty()) {
        System.out.println("Cola vacia. Nada que mostrar.");
        return;
    }
    System.out.print("Frente de la Cola -> ");
    lista.showlist();
}
}
```

Pila.java

```
public class Pila<T> {
    private ListaEnlazada<T> lista;

    public Pila() {
        this.lista = new ListaEnlazada<>(1);
    }

    public boolean isEmpty() {
        return lista.isEmpty();
    }

    public void push(T data) {
        lista.addFirst(data);
    }

    public T pop() {
        if (isEmpty()) {
            System.out.println("Pila vacia");
            return null;
        }
        return lista.removeFirst();
    }

    public T peek() {
        if (isEmpty()) {
            System.out.println("Pila vacia");
            return null;
        }
        return lista.getFirst();
    }

    public int getSize() {
        return lista.size;
    }
}
```

```
    }

    public void mostrarPila() {
        if (isEmpty()) {
            System.out.println("Pila vacia");
            return;
        }
        System.out.print("Tope de la Pila -> ");
        lista.showlist();
    }
}
```

App.java

```
import java.util.Scanner;

public class App {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        while (true) {
            System.out.println("\n-- Menu Principal --");
            System.out.println("1. Ejemplos de tipos de datos y listas");
            System.out.println("2. Sistema de gestion de contactos");
            System.out.println("3. Estructura de pilas y colas");
            System.out.println("4. Salir");
            System.out.print("Elige una opcion: ");

            int opmenu = scanner.nextInt();
            scanner.nextLine();

            if (opmenu == 1) {
                System.out.println("Modulo de Tipos de Datos (Asumiendo que ya tienes esta clase).");
            } else if (opmenu == 2) {
                menuContactos(scanner);
            } else if (opmenu == 3) {
                menuPilasColas(scanner);
            } else if (opmenu == 4) {
                System.out.println("Saliendo...");
                break;
            } else {
                System.out.println("Opcion no valida");
            }
        }
        scanner.close();
    }

    public static void menuContactos(Scanner scanner) {
        System.out.println("\n(Ingresando al modulo de contactos...)");
    }
}
```

```

public static void menuPilasColas (Scanner scanner) {
    Cola<Proceso> queueCPU = new Cola<>();
    Pila<String> pilaLlamadas = new Pila<>();

    int opcion;
    do {
        System.out.println("\n-- Simulador de SO: Pilas y Colas --");
        System.out.println("1. [Queue] Registrar un nuevo proceso
[Enqueue]");
        System.out.println("2. [Queue] Ejecutar siguiente proceso
[Dequeue]");
        System.out.println("3. [Queue] Ver proceso al frente [Peek]");
        System.out.println("4. [Queue] Mostrar cola de procesos");
        System.out.println("5. [Pila] Simular llamada a funcion
[Push]");
        System.out.println("6. [Pila] Retomar la funcion actual
[Pop]");
        System.out.println("7. [Pila] Ver funcion en tope de pila
[Peek]");
        System.out.println("8. [Pila] Mostrar pila de llamadas");
        System.out.println("9. Regresar al menuprincipal");
        System.out.print("Elige una opcion: ");

        opcion = scanner.nextInt();
        scanner.nextLine();

        switch (opcion) {
            case 1:
                System.out.print("ID del proceso: ");
                int id = scanner.nextInt();
                scanner.nextLine();
                System.out.print("Nombre del proceso: ");
                String nombre = scanner.nextLine();
                System.out.print("Usuario: ");
                String usuario = scanner.nextLine();
                System.out.print("Tiempo estimado: ");
                int tiempoEstimado = scanner.nextInt();
                scanner.nextLine();

                Proceso nuevoProceso = new Proceso(id, nombre, usuario,
tiempoEstimado);
                queueCPU.enqueue(nuevoProceso);
                System.out.println("Proceso registrado exitosamente.");
                break;
            case 2:
                System.out.println("Ejecutando proceso...");
                Proceso ejecutado = queueCPU.dequeue();
                if (ejecutado != null) {
                    System.out.println("Procesando tarea: " +
ejecutado);
                } else {
                    System.out.println("No hay procesos pendientes.");
                }
        }
    }
}

```

```
        break;
    case 3:
        Proceso frente = queueCPU.peek();
        if (frente != null) {
            System.out.println("Primer proceso a ejecutar: " +
frente);
        } else {
            System.out.println("No hay procesos en la cola.");
        }
        break;
    case 4:
        System.out.println("Cola de procesos actual: ");
        queueCPU.mostrarCola();
        break;
    case 5:
        System.out.print("Ingresa el nombre de la funcion: ");
        String funcion = scanner.nextLine();
        pilaLlamadas.push(funcion);
        System.out.println("Funcion apilada correctamente.");
        break;
    case 6:
        System.out.println("Retornando la funcion...");
        String terminada = pilaLlamadas.pop();
        if (terminada != null) {
            System.out.println("Funcion terminada: " +
terminada);
        } else {
            System.out.println("No hay funciones activas en la
pila.");
        }
        break;
    case 7:
        String tope = pilaLlamadas.peek();
        if (tope != null) {
            System.out.println("Funcion actual en el tope: " +
tope);
        } else {
            System.out.println("Pila vacia, nada que
mostrar.");
        }
        break;
    case 8:
        System.out.println("Estado de la Pila de llamadas:");
        pilaLlamadas.mostrarPila();
        break;
    case 9:
        System.out.println("Regresando al menu principal...");
        break;
    default:
        System.out.println("Opcion no disponible.");
        break;
    }
} while (opcion != 9);
```

```
}  
}
```

Código corriendo

```

$ java App.java

-- Menu Principal --
1. Ejemplos de tipos de datos y listas
2. Sistema de gestion de contactos
3. Estructura de pilas y colas
4. Salir
Elige una opcion: 3

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 1
ID del proceso: 1
Nombre del proceso: proceso1
Usuario: Erick
Tiempo estimado: 15
Proceso registrado exitosamente.

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 2

2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 3
Primer proceso a ejecutar: Id: 1| Proceso: proceso1| Usuario: Erick| tiempo estimado: 15

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 4
Cola de procesos actual:
Frente de la Cola → Id: 1| Proceso: proceso1| Usuario: Erick| tiempo estimado: 15 → Id: 2| Proceso: proceso2| Usuario: Erick| tiempo estimado: 15 → null

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion:

```

```

1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 2
Ejecutando proceso...
Procesando tarea: Id: 1| Proceso: proceso1| Usuario: Erick| tiempo estimado: 15

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 3
Primer proceso a ejecutar: Id: 2| Proceso: proceso2| Usuario: Erick| tiempo estimado: 15

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion:

2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 5
Ingresa el nombre de la funcion: funcion2
Funcion apilada correctamente.

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 5
Ingresa el nombre de la funcion: funcion3
Funcion apilada correctamente.

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: █

```

```

1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 7
Funcion actual en el tope: funcion3

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 8
Estado de la Pila de llamadas:
Tope de la Pila → funcion3 → funcion2 → funcion1 → null

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion:

```

```

2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 6
Retornando la funcion...
Funcion terminada: funcion3

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: 8
Estado de la Pila de llamadas:
Tope de la Pila → funcion2 → funcion1 → null

-- Simulador de S0: Pilas y Colas --
1. [Queue] Registrar un nuevo proceso [Enqueue]
2. [Queue] Ejecutar siguiente proceso [Dequeue]
3. [Queue] Ver proceso al frente [Peek]
4. [Queue] Mostrar cola de procesos
5. [Pila] Simular llamada a funcion [Push]
6. [Pila] Retomar la funcion actual [Pop]
7. [Pila] Ver funcion en tope de pila [Peek]
8. [Pila] Mostrar pila de llamadas
9. Regresar al menu principal
Elige una opcion: █

```